GitHub Automation using Jenkins

Objective

To automate the process of **building and executing a Java program using Jenkins whenever code is pushed to GitHub**.

This is achieved using:

- **GitHub Repository**
- **Jenkins**
- **ngrok (to expose local Jenkins server)**
- **GitHub Webhook**

When a developer pushes code to GitHub, Jenkins automatically pulls the code, compiles it, and executes the program.

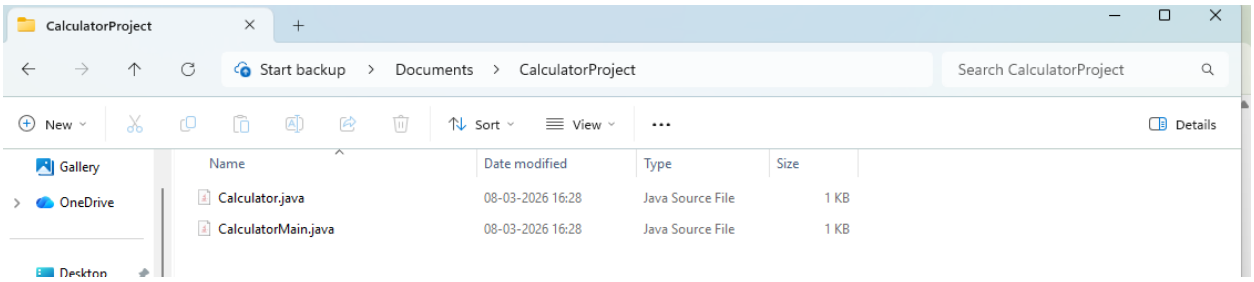
Step 1: Create Java Project

A Java project named **CalculatorProject** was created containing two files:

- `Calculator.java`
- `CalculatorMain.java`

These files implement basic arithmetic operations such as:

- Addition
- Subtraction
- Multiplication
- Division



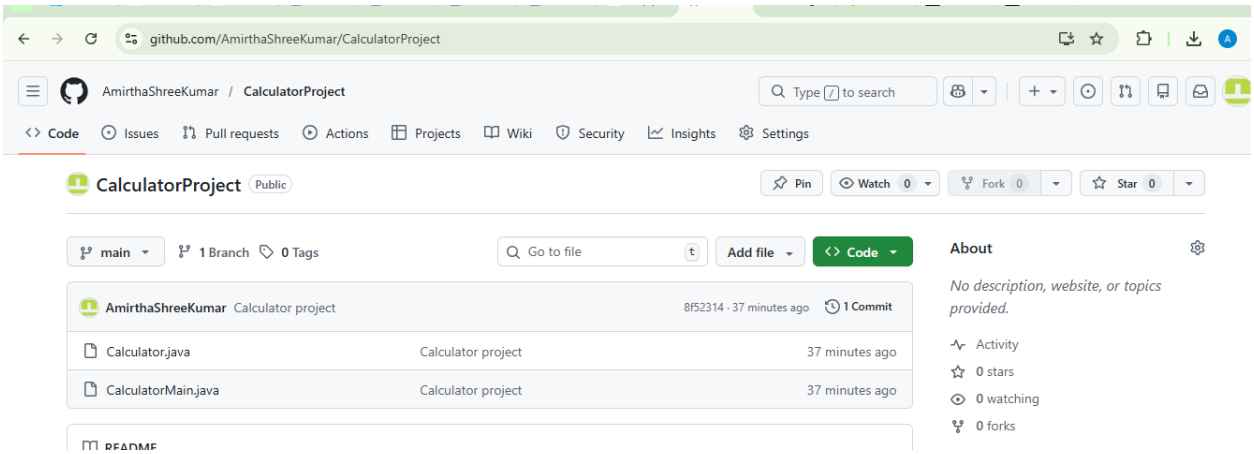
Step 2: Push Project to GitHub

The project folder was pushed to GitHub using Git Bash.

Commands used:

```
git init
git add .
git commit -m "Calculator project"
git branch -M main
git remote add origin
https://github.com/AmirthaShreeKumar/CalculatorProject.git
git push -u origin main
```

This uploaded the project files to the GitHub repository.



Step 3: Start Jenkins Server

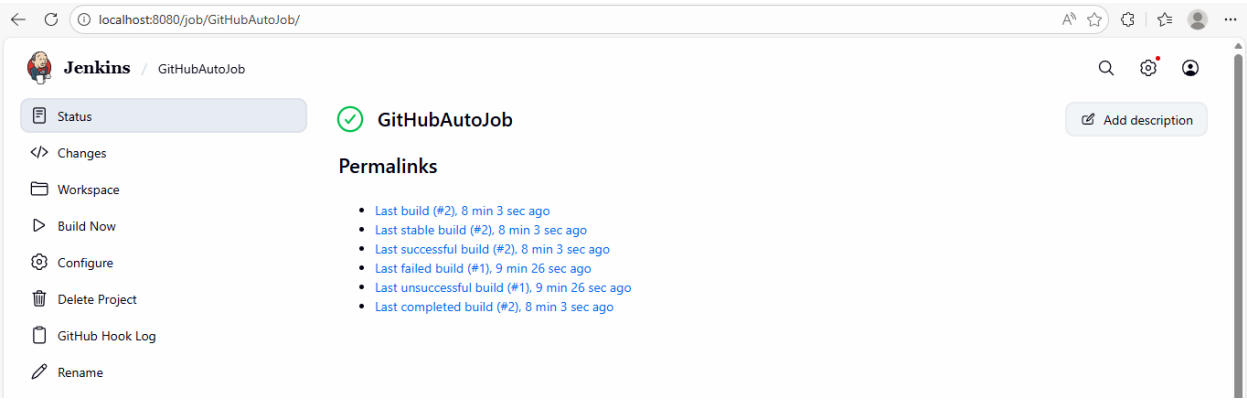
Jenkins was started locally and accessed through:

http://localhost:8080

A new Jenkins job was created.

Steps followed:

```
Jenkins Dashboard
→ New Item
→ Freestyle Project
→ Job name: GitHubAutoJob
```



Step 4: Connect Jenkins with GitHub Repository

Inside the Jenkins job configuration:

```
Configure
→ Source Code Management
```

→ Git

The GitHub repository URL was provided.

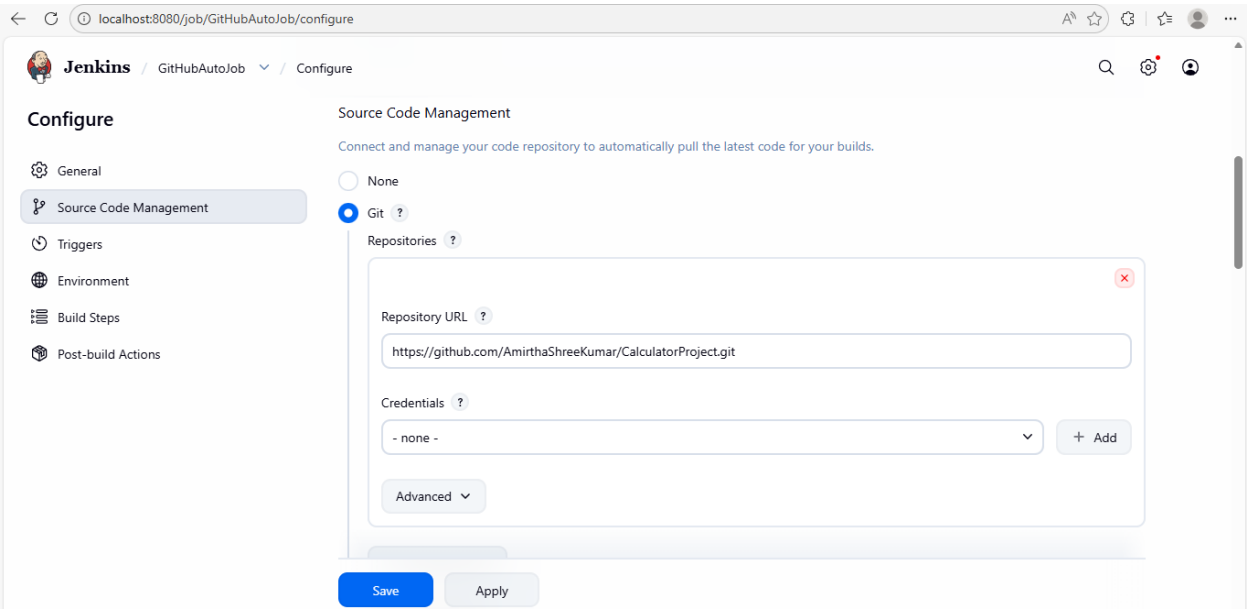
Example:

https://github.com/AmirthaShreeKumar/CalculatorProject.git

Branch specified:

*/main

This allows Jenkins to fetch code from the GitHub repository.



Step 5: Add Build Command

A build step was added so Jenkins knows how to run the Java program.

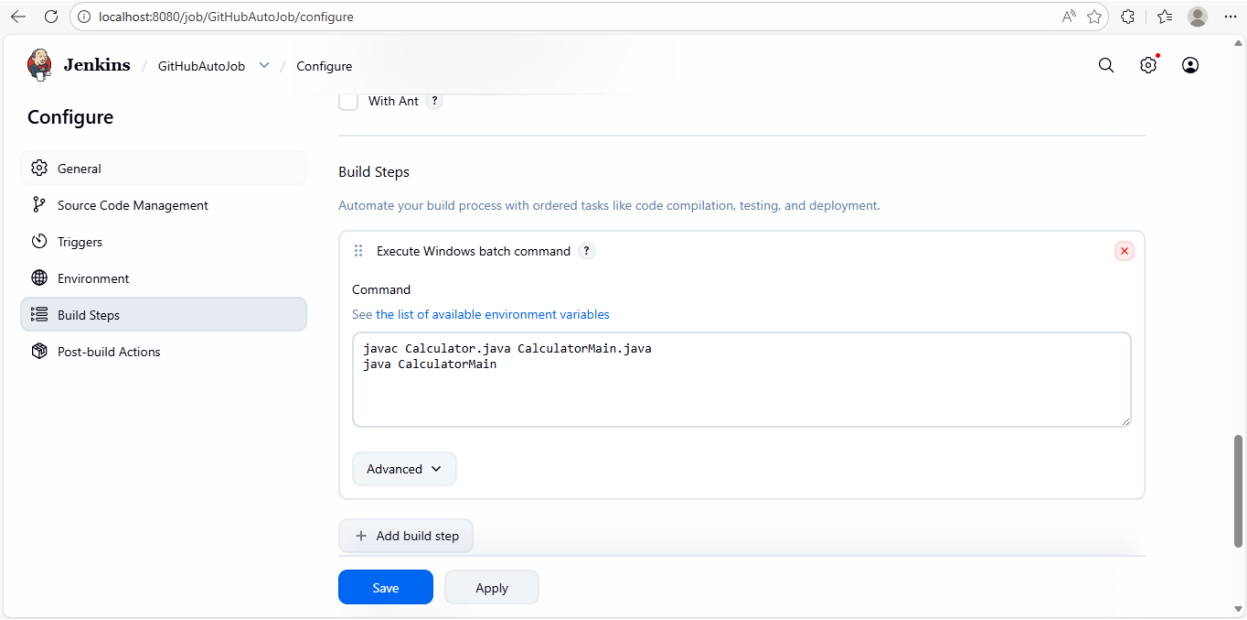
Steps:

```
Configure
→ Build
→ Add Build Step
→ Execute Windows batch command
```

Command used:

```
javac Calculator.java CalculatorMain.java
java CalculatorMain
```

This command compiles and executes the Java program.



Step 6: Enable Automatic Trigger

To allow GitHub to trigger Jenkins automatically:

```
Configure
→ Build Triggers
→ GitHub hook trigger for GITScm polling
```

This enables Jenkins to listen for GitHub webhook events.

Step 7: Expose Jenkins using ngrok

Since Jenkins runs locally, ngrok was used to create a public URL.

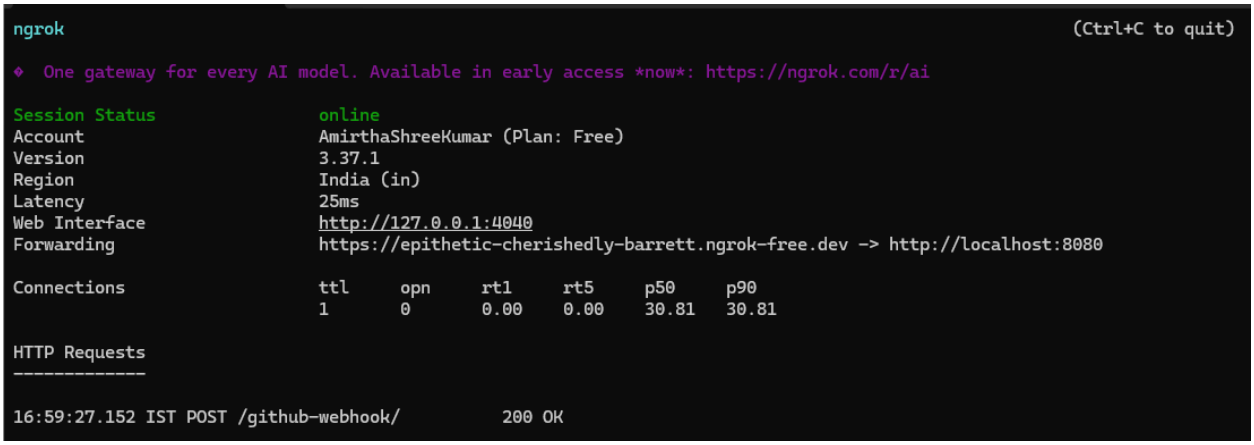
Command executed:

```
ngrok http 8080
```

ngrok generated a forwarding URL such as:

https://abcd1234.ngrok-free.app

This URL allows GitHub to communicate with Jenkins.



Step 8: Configure GitHub Webhook

In the GitHub repository:

```
Settings
→ Webhooks
→ Add Webhook
```

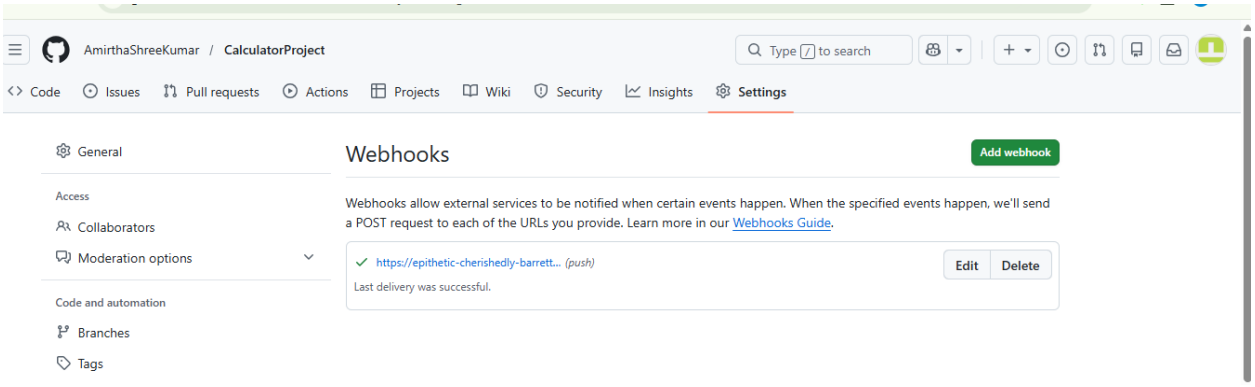
Payload URL:

```
https://epithetic-cherishedly-barrett.ngrok-free.dev/github-wehbook
```

Content Type:

```
application/json
```

This webhook sends a notification to Jenkins whenever code is pushed to GitHub.



Step 9: Build Execution in Jenkins

After pushing code to GitHub:

```
git add .
git commit -m "Test automation"
```

```
git push
```

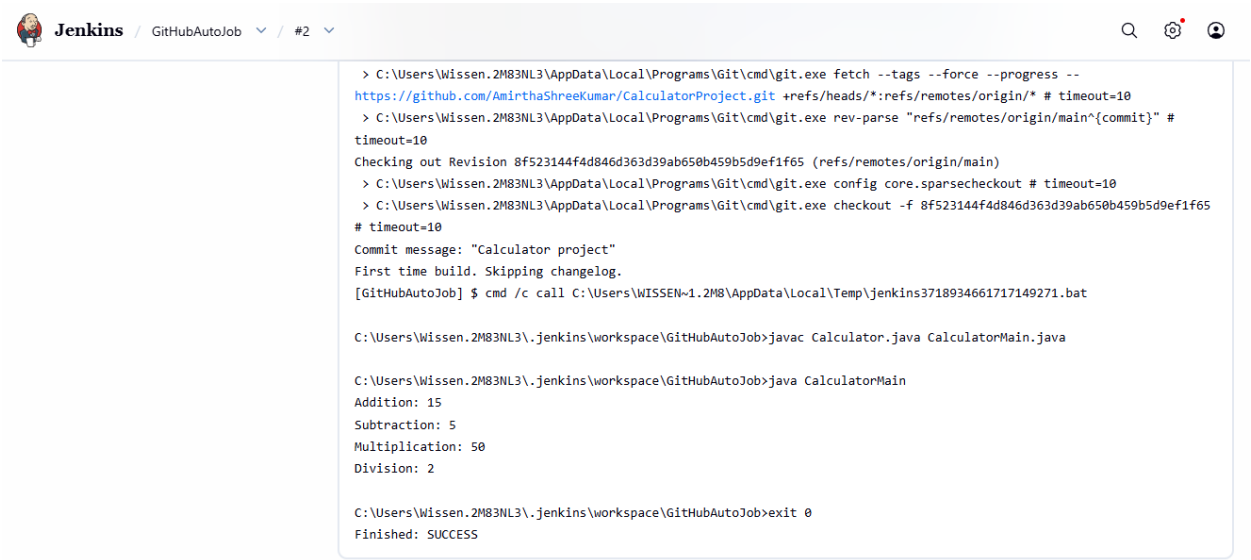
GitHub triggers Jenkins through the webhook.

Jenkins performs the following steps automatically:

1. Pulls latest code from GitHub
2. Compiles the Java files
3. Executes the program

Console output:

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
Finished: SUCCESS
```



localhost:8080/job/GitHubAutoJob/2/consoleText

```
Started by user Amirtha Shree K
Running as SYSTEM
Building in workspace C:\Users\Wissen.2M83NL3\.jenkins\workspace\GitHubAutoJob
The recommended git tool is: NONE
No credentials specified
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe rev-parse --resolve-git-dir C:\Users\Wissen.2M83NL3\.jenkins\workspace\GitHubAutoJob\.git # timeout=10
Fetching changes from the remote Git repository
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe config remote.origin.url https://github.com/AmirthaShreeKumar/CalculatorProject.git # timeout=10
Fetching upstream changes from https://github.com/AmirthaShreeKumar/CalculatorProject.git
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe fetch --tags --force --progress -- https://github.com/AmirthaShreeKumar/CalculatorProject.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 8f523144f4d846d363d39ab650b459b5d9ef1f65 (refs/remotes/origin/main)
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe config core.sparsecheckout # timeout=10
 > C:\Users\Wissen.2M83NL3\AppData\Local\Programs\Git\cmd\git.exe checkout -f 8f523144f4d846d363d39ab650b459b5d9ef1f65 # timeout=10
Commit message: "Calculator project"
First time build. Skipping changelog.
[GitHubAutoJob] $ cmd /c call C:\Users\WISSEN~1.2M8\AppData\Local\Temp\jenkins3718934661717149271.bat

C:\Users\Wissen.2M83NL3\.jenkins\workspace\GitHubAutoJob>javac Calculator.java CalculatorMain.java

C:\Users\Wissen.2M83NL3\.jenkins\workspace\GitHubAutoJob>java CalculatorMain
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2

C:\Users\Wissen.2M83NL3\.jenkins\workspace\GitHubAutoJob>exit 0
Finished: SUCCESS
```

Final Workflow

The automation pipeline works as follows:

```
Developer pushes code
        ↓
GitHub repository updated
        ↓
GitHub Webhook triggered
        ↓
ngrok forwards request to Jenkins
        ↓
Jenkins pulls latest code
        ↓
Jenkins compiles and runs program
```

Result

The GitHub automation using Jenkins was successfully implemented.

Whenever code is pushed to GitHub, Jenkins automatically compiles and executes the Java program.